



VISUAL BASIC C# PROGRAMMING (330)

STATE – 2020

Production Portion:

Program 1: PDF Manager Application _____ (450 points)

TOTAL POINTS _____ ***(450 points)***

Failure to adhere to any of the following rules will result in disqualification:

- 1. Contestant must hand in this test booklet and all printouts. Failure to do so will result in disqualification.**
- 2. No equipment, supplies, or materials other than those specified for this event are allowed in the testing area. No previous BPA tests and/or sample tests or facsimile (handwritten, photocopied, or keyed) are allowed in the testing area.**
- 3. Electronic devices will be monitored according to ACT standards.**

No more than ten (10) minutes orientation

No more than 90 minutes testing time

No more than ten (10) minutes wrap-up

Property of Business Professionals of America.
May be reproduced only for use in the Business Professionals of America
Workplace Skills Assessment Program competition.

PDF Manager Application

Textbooks are becoming less and less common in our education system, with more teachers and students preferring the use of e-books (PDFs for simplicity). For this exercise, you will create a Visual Basic or C# Windows Form Application that is suitable for managing user login, the PDFs a user owns, viewing those PDFs, and purchasing additional PDFs. XML Serialization will be used for a database to store both PDF and user information.

Requirements:

1. You must create a C#/VB Windows Form Application named CS_330_ContestantNumber, where ContestantNumber is your BPA assigned contestant number (including dashes). For example, CS_330_01_2345_6789. If you are using VB, use VB instead of CS in the application name.
2. Your contestant number must appear as a comment at the top of the main source code file.
3. On application start, a login form shall appear.
 - a. This must be done using a different form than the main application.
 - b. When started, both the user and pdf databases shall be loaded from files “pdfs.xml” and “users.xml”. The databases can be represented as lists of User and Pdf objects.
 - c. If a proper entry is made (defined as username and password matching an existing user’s information), the Login form shall disappear and the main application shall launch.
 - d. The login form shall close after the main application closes.
 - e. The login form, with proper entry, shall pass both the user and pdf databases as well as the integer index the logged in user can be found at.
4. The login form must not have minimize or maximize buttons in the Form Handle.
5. The login form must have text “Login” for its title.
6. The login form must have textboxes for Username and Password entry, named “userBox” and “passBox” respectively and with suitable labels named “userLbl” and “passLbl”.
 - a. “userBox” shall have focus immediately upon form launch and pressing tab shall shift focus to “passBox”.
 - b. Pressing enter shall invoke a button “loginBtn” with text “Login” to check the user’s entry.
7. If invalid user information is entered, display a warning with message “Invalid login.”
8. Two classes must be created – Pdf and User – to store the information from the two XML databases.
 - a. The Pdf class shall have integer Id, string File, and double Cost public properties
 - b. The User class shall have string Name, string Password, integer list Pdfs, and double Money public properties
 - i. The Pdfs integer list shall be used to relate to a Pdf objects Id field and shall be used to mark which Pdfs a user has access to.

- c. The User class shall have static functions for saving and loading, while the Pdf class shall only have a static load function, named “SaveUsers”, “LoadUsers”, and “LoadPdfs” respectively.
 - i. The Save functions shall take in the current database (list of objects)
 - ii. The Load functions shall attempt to load in the database file, but if not present shall create a new database file with sample data. The loaded database shall then be returned. The specifications for sample data are provided in the included pdfs.xml and users.xml.
 - iii. Both sets of functions shall have suitable error messages when either fails.
 - d. Suitable constructors shall be created for both classes.
 - i. An empty constructor.
 - ii. A constructor that accepts values for each public property.
 - e. The User class shall have a “PurchasePdf” public function with a Pdf object as a parameter.
 - i. The function shall check if the user has enough money for the selected pdf, and if so, deduct the cost while adding the pdf’s Id field to the Pdfs integer list of the user.
 - ii. Shall return a bool value indicating if purchase was successful.
9. Loading and saving of User and Pdf databases shall be done using XML serialization. The code for performing generic XML serialization is included in the provided Serialization class and shall be used by the “LoadUsers” and “LoadPdfs” functions.
10. Main Form on close must ask for confirmation to close, with the Title of the confirmation box being “Confirm” and the text being “Are you sure you wish to exit the PDF Manager?”
- a. Login form shall be allowed to close without asking confirmation.
 - b. When the main form successfully closes, the SaveUsers function should be called to update the database file.
11. The main form must have text “PDF Manager” for its title.
12. The main form must have a tab control containing three tabs.
- a. The tab control must be named “pdfManagerTabs”, with Dock set to Fill.
 - b. The first tab must be named “pdfSelectPage” with text “My PDFs”. It must contain a button with text “Select PDF” and named “selectPdfBtn” with dock set to Bottom as well as a ListBox named “ownedPdfList” with dock set to Fill.
 - c. The second tab must be named “currentPdfPage” with text “Current PDF”. It must contain a WebBrowser named “pdfViewer” with dock set to Fill.
 - d. The third tab must be named “newPdfPage” with text “Get New PDFs”. It must contain a button with text “Purchase PDF” and named “buyPdfBtn” with dock set to Bottom as well as a ListBox named “buyablePdfList” with dock set to Fill.
13. When the main form is constructed, the current user index shall be used in conjunction with the two passed databases to populate the buyablePdfList and ownedPdfList controls.
- a. Any pdfs with Ids matching what is in the current user’s Pdfs list shall be added to the ownedPdfList control, otherwise it shall be added to the buyablePdfList control.

- b. This should be done using a suitable function that also clears these two lists prior to population.
 - c. When a pdf is purchased by the current user, this action should be performed again to refresh the list.
- 14. The sample PDF files shall be stored in the same directory as the application. **The contestant will need to move them to this directory after the application is built the first time.**
- 15. When the selectPdfBtn is pressed, the following occurs:
 - a. It is verified a single item is selected in the ownedPdfList.
 - b. It is verified the selected file exists.
 - c. If both conditions are present, the pdfViewer WebBrowser control navigates to that file to load the file, and the TabControl switches the selected tab to the viewing page.
 - d. A failure at any of these steps shall result in an error message.
- 16. When the buyPdfBtn is pressed, the following occurs:
 - a. It is verified a single item is selected in the buyablePdfList.
 - b. It is verified the selected file exists.
 - c. The Pdf object is retrieved from the database using the string stored by the buyablePdfList selected item.
 - d. The current user attempts to purchase the item using the PurchasePdf function.
 - e. A failure at any of these steps shall result in an error message.

Sample Pictures:

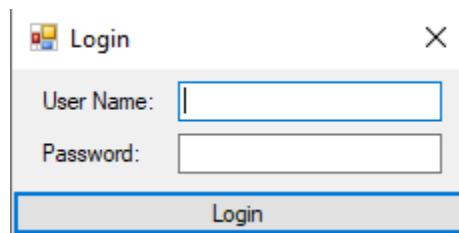


Figure 1: The login form

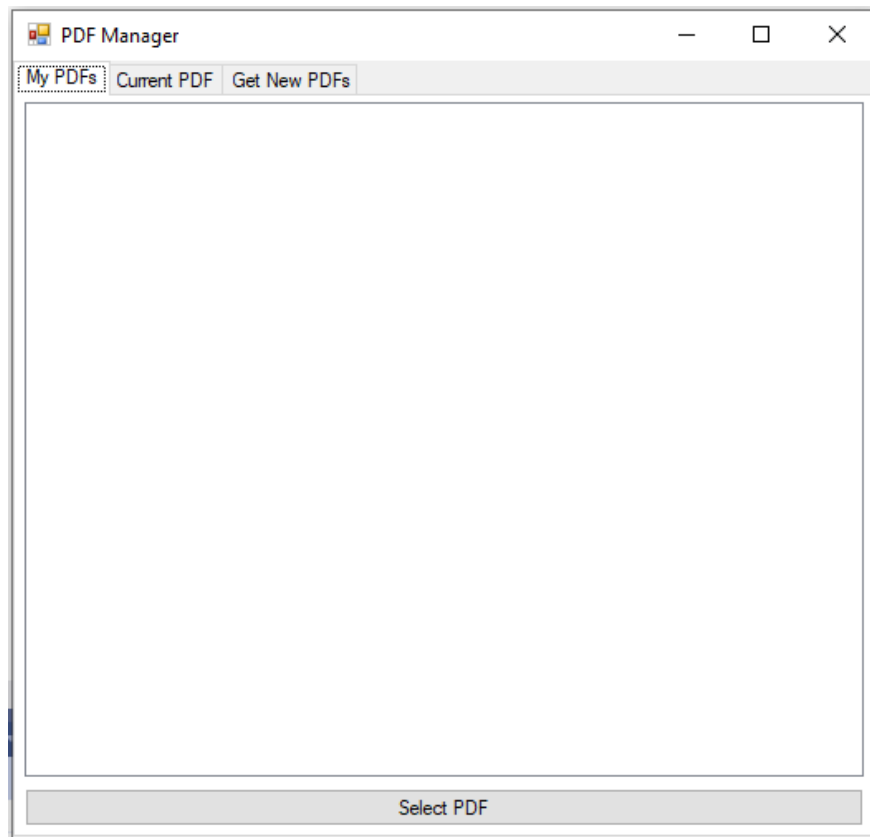


Figure 2: The PDF Manager form with no purchased PDFs.

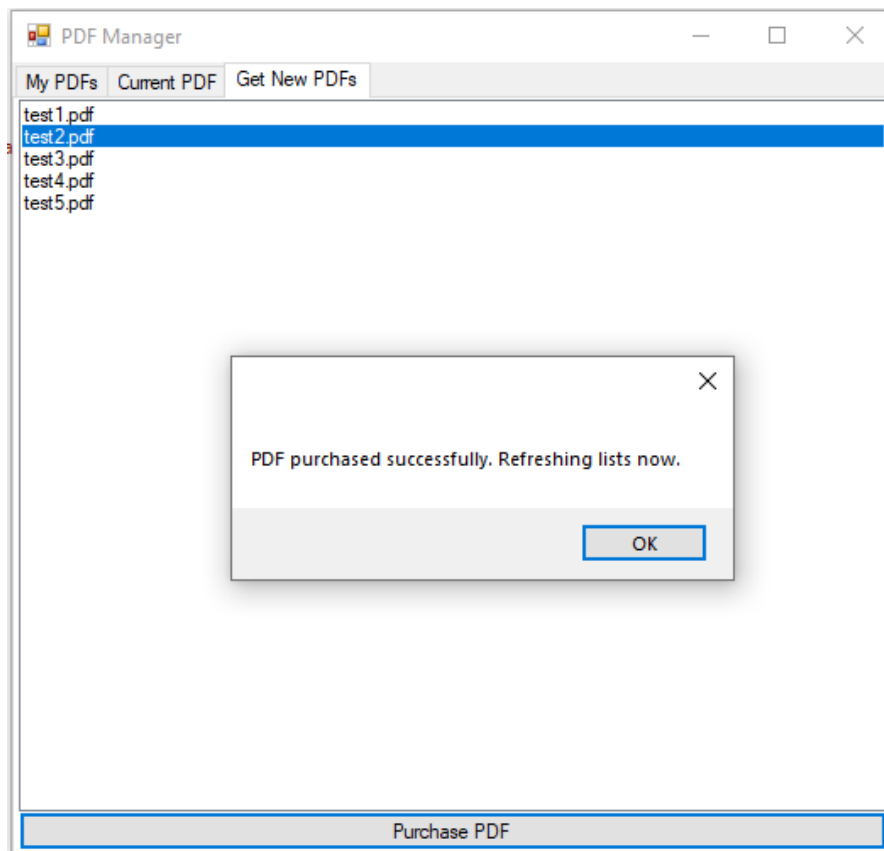


Figure 3: Successful purchasing of test2.pdf. After hitting OK, test2.pdf is removed from this list.

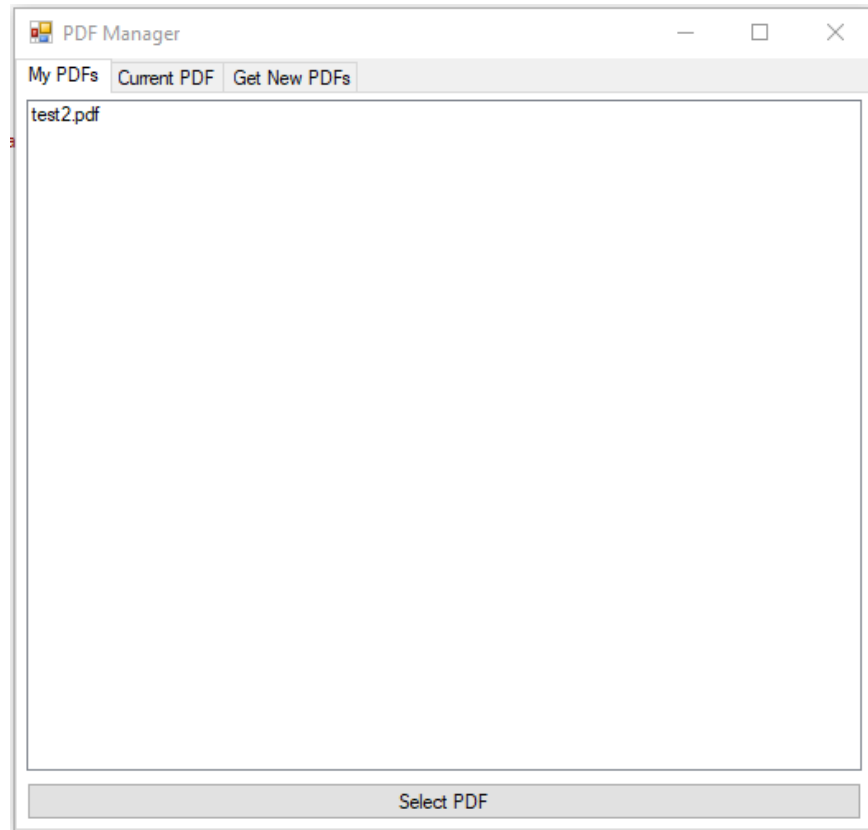


Figure 4: Successful purchasing of test2.pdf, it is shown in My PDFs.

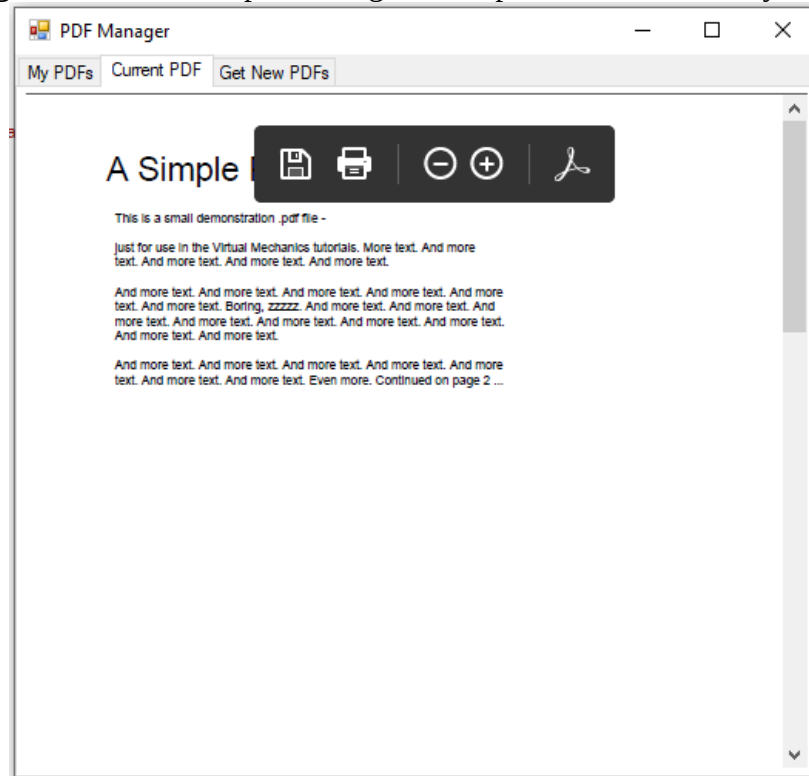


Figure 5: After pressing Select PDF, the PDF is loaded using the pdfViewer WebBrowser and the active tab page is switched to Current PDF.

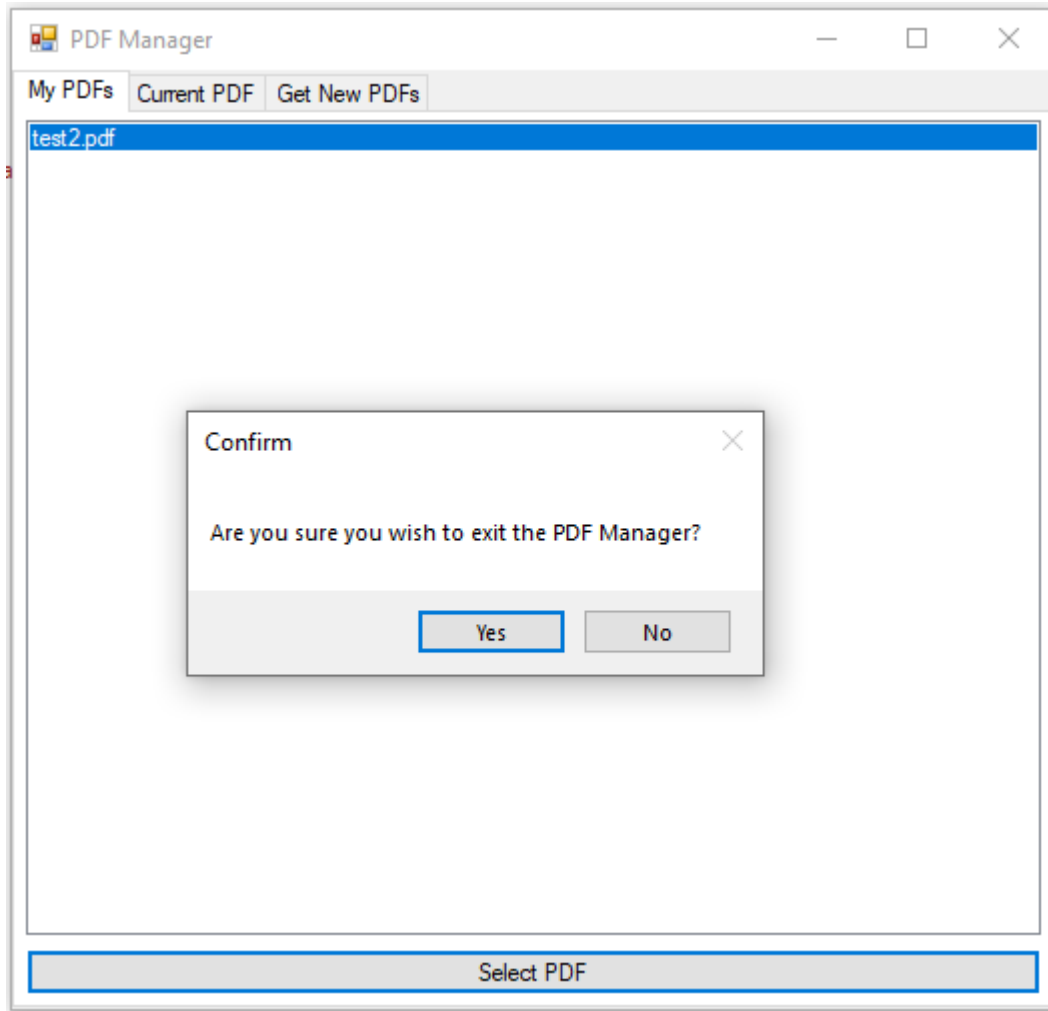


Figure 6: The confirmation window. This should be opened when Exit is pressed. If “Yes” is clicked, the application should close, otherwise return to the main form.

You will have 90 minutes to complete your work.

Your name and/or school name should NOT appear on any work you submit for grading.

Submit a copy your entire solution/project to the flash drive provided. You must submit your entire solution/project so that the graders may open your project to review the source code and/or build and execute your solution/project. **Submissions that do not contain source code will not be graded.**

Development Standards

- Standard name prefixes must be utilized for variables.
- All subroutines, functions, and methods must be documented with comments explaining the purpose of the method, the input parameters (if any), and the output (if any).

Your application will be graded on the following criteria:

Solution and Project

The project is present on the flash drive	_____ 10 pts
The project is named according to the naming conventions	_____ 10 pts

Program Execution

Code copied to USB drive and program runs from USB	_____ 20 pts
--	--------------

If the program does not execute, then the remaining items in this section receive a score of zero.

The main form asks for confirmation on close, login does not	_____ 15 pts
Database state is saved on closing main form	_____ 20 pts
Database state is loaded on opening login form from pdfs.xml and users.xml	_____ 20 pts
Textboxes are present to enter user information in login form	_____ 10 pts
Login successfully pulls correct user information from database	_____ 25 pts
Main form is initialized with current user and database after login	_____ 25 pts
Login form properly disappears after successful login	_____ 10 pts
Main form has tab control with suitable pages	_____ 20 pts
Each page has specified controls with similar layout	_____ 20 pts
On main form loading and purchase of Pdf, PDF lists are refreshed	_____ 25 pts
My PDFs page is refreshed with owned PDFs	_____ 10 pts
Get New PDFs page is refreshed only with not owned PDFs	_____ 10 pts
Selecting in My PDFs results in transition to Current PDF and loading the file	_____ 30 pts
PDF purchasing if user has enough money and adds PDF to owned list	_____ 20 pts
Error messages are displayed according to specifications	_____ 20 pts

Source Code Review

Code is commented at the top, for each function, and as needed	_____ 10 pts
Code uses reasonable and consistent variable naming conventions	_____ 10 pts
Application makes use of two separate forms	_____ 20 pts
Provided Serialization class is used for XML Serialization	_____ 20 pts
User class is created and has specified variables and functions	_____ 10 pts
Pdf class is created and has specified variables and functions	_____ 10 pts
Suitable constructors are present for both User and Pdf classes	_____ 10 pts
Controls are named as described in requirements	_____ 20 pts
WebBrowser control is used to load current PDF	_____ 20 pts

Total Points: 450 pts

```
User.cs
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Windows.Forms;

namespace kindle_clone
{
    public class User
    {
        public string Name { get; set; }
        public string Password { get; set; }
        public List<int> Pdfs { get; set; }
        public double Money { get; set; }

        public User()
        {
            Name = "";
            Password = "";
            Pdfs = new List<int>();
            Money = 0.00;
        }

        public User(string name, string password, List<int> pdfs, double money)
        {
            Name = name;
            Password = password;
            Pdfs = pdfs;
            Money = money;
        }

        public bool PurchasePdf(Pdf pdf)
        {
            bool success = false;

            if (Money >= pdf.Cost)
            {
                success = true;
                Money -= pdf.Cost;
                Pdfs.Add(pdf.Id);
            }

            return success;
        }

        public static void SaveUsers(List<User> users)
        {
            try
            {
                Serialization.Serialize<List<User>>(users, "users.xml");
            }
            catch
            {
                MessageBox.Show("Error: Saving User List Failed.");
            }
        }
    }
}
```

```

        public static List<User> LoadUsers()
        {
            List<User> users;

            try
            {
                users = Serialization.Deserialize<List<User>>("users.xml");
            }
            catch
            {
                MessageBox.Show("Error: No User database file - creating new database with sample data.");
                users = new List<User>();
                users.Add(new User("Bruce", "qwe", new List<int>(), 250.00));
                users.Add(new User("Joseph", "asdf", new List<int>(), 150.00));
                users.Add(new User("George", "asdf", new List<int>(), 150.00));
                Serialization.Serialize<List<User>>(users, "users.xml");
            }

            return users;
        }
    }
}
Login.cs
using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Drawing;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Windows.Forms;

namespace kindle_clone
{
    public partial class Login : Form
    {
        List<User> users;
        List<Pdf> pdfs;

        public Login()
        {
            InitializeComponent();

            users = User.LoadUsers();
            pdfs = Pdf.LoadPdfs();
        }

        private void loginBtn_Click(object sender, EventArgs e)
        {
            string username = userBox.Text;
            string password = passBox.Text;

            for (int i = 0; i < users.Count; i++)
            {
                if (users[i].Name == username && users[i].Password == password)
                {
                    this.Visible = false;
                }
            }
        }
    }
}

```

```
        new Form1(users, pdfs, i)..ShowDialog();  
        this.Close();  
        break;  
    }  
}  
  
if (this.Visible)  
{  
    MessageBox.Show("Invalid login.");  
}  
}  
}
```

Pdf.cs

```
using System;  
using System.Collections.Generic;  
using System.Linq;  
using System.Text;  
using System.Threading.Tasks;  
using System.Windows.Forms;
```

namespace kindle_clone
{
 public class Pdf
 {
 public int Id { get; set; }
 public string File { get; set; }
 public double Cost { get; set; }

public Pdf()
{
 Id = 0;
 File = "";
 Cost = 0.00;
}

public Pdf(int id, string file, double cost)
{
 Id = id;
 File = file;
 Cost = cost;

}

public static List<Pdf> LoadPdfs()
{
 List<Pdf> pdfs;

 try
 {
 pdfs = Serialization.Deserialize<List<Pdf>>("pdfs.xml");
 }
 catch
 {
 MessageBox.Show("Error: No Pdf database file - creating new database with sample data.");
 pdfs = new List<Pdf>();
 pdfs.Add(new Pdf(1, "test1.pdf", 15.00));
 pdfs.Add(new Pdf(2, "test2.pdf", 35.00));
 pdfs.Add(new Pdf(3, "test3.pdf", 10.00));
 pdfs.Add(new Pdf(4, "test4.pdf", 75.00));
 }
}

```
        pdfs.Add(new Pdf(5, "test5.pdf", 45.00));
        Serialization.Serialize<List<Pdf>>(pdfs, "pdfs.xml");
    }

    return pdfs;
}
}
}
Form1.cs
using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Drawing;
using System.IO;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Windows.Forms;

namespace kindle_clone
{
    public partial class Form1 : Form
    {
        List<User> users;
        List<Pdf> pdfs;

        int userIndex;

        public Form1(List<User> newUsers, List<Pdf> newPdfs, int newUserIndex)
        {
            InitializeComponent();

            users = newUsers;
            pdfs = newPdfs;
            userIndex = newUserIndex;

            refreshLists();
        }

        private void refreshLists()
        {
            ownedPdfList.Items.Clear();
            buyablePdfList.Items.Clear();

            foreach (Pdf pdf in pdfs)
            {
                bool found = false;

                foreach (int i in users[userIndex].Pdfs)
                {
                    found |= (i == pdf.Id);
                }

                if (found)
                {
                    ownedPdfList.Items.Add(pdf.File);
                }
                else
            }
        }
    }
}
```

```

        {
            buyablePdfList.Items.Add(pdf.File);
        }
    }
}

private void selectPdfBtn_Click(object sender, EventArgs e)
{
    if (ownedPdfList.SelectedItems.Count == 1)
    {
        if (File.Exists(ownedPdfList.SelectedItem.ToString()))
        {
            try
            {
                pdfViewer.Navigate(@"file:/// + Directory.GetCurrentDirectory() + "/" +
ownedPdfList.SelectedItem.ToString());
                pdfManagerTabs.SelectedTab = currentPdfPage;
            }
            catch
            {
                MessageBox.Show("Unable to load selected PDF.");
            }
        }
    }
    else
    {
        MessageBox.Show("Ensure a single PDF is selected.");
    }
}

private void buyPdfBtn_Click(object sender, EventArgs e)
{
    if (buyablePdfList.SelectedItems.Count == 1)
    {
        if (File.Exists(buyablePdfList.SelectedItem.ToString()))
        {
            bool found = false;

            foreach (Pdf pdf in pdfs)
            {
                if (pdf.File == buyablePdfList.SelectedItem.ToString())
                {
                    if (users[userIndex].PurchasePdf(pdf))
                    {
                        MessageBox.Show("PDF purchased successfully. Refreshing lists now.");
                    }
                    else
                    {
                        MessageBox.Show("Cannot purchase PDF.");
                    }
                    found = true;
                }
            }

            if (found)
            {
                refreshLists();
            }
            else

```

```
        {
            MessageBox.Show("Unable to find PDF in library.");
        }
    }
    else
    {
        MessageBox.Show("Unable to find selected PDF.");
    }
}
else
{
    MessageBox.Show("Ensure a single PDF is selected.");
}
}
private void Form1_FormClosing(object sender, FormClosingEventArgs e)
{
    if (MessageBox.Show("Are you sure you wish to exit the PDF Manager?", "Confirm",
        MessageBoxButtons.YesNo) == System.Windows.Forms.DialogResult.Yes)
    {
        User.SaveUsers(users);
    }
    else
    {
        e.Cancel = true;
    }
}
}
```

FORM1.vb

Imports System

Imports System.Collections.Generic

Imports System.ComponentModel

Imports System.Data

Imports System.Drawing

Imports System.IO

Imports System.Linq

Imports System.Text

Imports System.Threading.Tasks

Imports System.Windows.Forms

Namespace kindle_clone

Public Partial Class Form1

Inherits Form

Private users As List(Of User)

Private pdfs As List(Of Pdf)

Private userIndex As Integer

Public Sub New(ByVal newUsers As List(Of User), ByVal newPdfs As List(Of Pdf), ByVal newUserIndex As Integer)

InitializeComponent()

users = newUsers

pdfs = newPdfs

userIndex = newUserIndex

refreshLists()

End Sub

Private Sub refreshLists()

ownedPdfList.Items.Clear()

buyablePdfList.Items.Clear()

For Each pdf As Pdf In pdfs

Dim found As Boolean = False

For Each i As Integer In users(userIndex).Pdfs

found = found Or (i = pdf.Id)

Next

If found Then

ownedPdfList.Items.Add(pdf.File)

Else

buyablePdfList.Items.Add(pdf.File)

End If

Next

End Sub

Private Sub selectPdfBtn_Click(ByVal sender As Object, ByVal e As EventArgs)

If ownedPdfList.SelectedItems.Count = 1 Then

If File.Exists(ownedPdfList.SelectedItem.ToString()) Then

Try

pdfViewer.Navigate("file:/// " & Directory.GetCurrentDirectory() & "/" &

ownedPdfList.SelectedItem.ToString())

pdfManagerTabs.SelectedTab = currentPdfPage

```
        Catch
            MessageBox.Show("Unable to load selected PDF.")
        End Try
    End If
Else
    MessageBox.Show("Ensure a single PDF is selected.")
End If
End Sub

Private Sub buyPdfBtn_Click(ByVal sender As Object, ByVal e As EventArgs)
    If buyablePdfList.SelectedItems.Count = 1 Then

        If File.Exists(buyablePdfList.SelectedItem.ToString()) Then
            Dim found As Boolean = False

            For Each pdf As Pdf In pdfs

                If pdf.File = buyablePdfList.SelectedItem.ToString() Then

                    If users(userIndex).PurchasePdf(pdf) Then
                        MessageBox.Show("PDF purchased successfully. Refreshing lists now.")
                    Else
                        MessageBox.Show("Cannot purchase PDF.")
                    End If

                    found = True
                End If
            Next

            If found Then
                refreshLists()
            Else
                MessageBox.Show("Unable to find PDF in library.")
            End If
        Else
            MessageBox.Show("Unable to find selected PDF.")
        End If
    Else
        MessageBox.Show("Ensure a single PDF is selected.")
    End If
End Sub

Private Sub Form1_FormClosing(ByVal sender As Object, ByVal e As FormClosingEventArgs)
    If MessageBox.Show("Are you sure you wish to exit the PDF Manager?", "Confirm",
        MessageBoxButtons.YesNo) = System.Windows.Forms.DialogResult.Yes Then
        User.SaveUsers(users)
    Else
        e.Cancel = True
    End If
End Sub
End Class
End Namespace

User.vb
Imports System
Imports System.Collections.Generic
Imports System.Linq
Imports System.Text
Imports System.Threading.Tasks
```

Imports System.Windows.Forms

Namespace kindle_clone

Public Class User

Public Property Name As String

Public Property Password As String

Public Property Pdfs As List(Of Integer)

Public Property Money As Double

Public Sub New()

 Name = ""

 Password = ""

 Pdfs = New List(Of Integer)()

 Money = 0.00

End Sub

Public Sub New(ByVal name As String, ByVal password As String, ByVal pdfs As List(Of Integer), ByVal money As Double)

 Name = name

 Password = password

 Pdfs = pdfs

 Money = money

End Sub

Public Function PurchasePdf(ByVal pdf As Pdf) As Boolean

 Dim success As Boolean = False

 If Money >= pdf.Cost Then

 success = True

 Money -= pdf.Cost

 Pdfs.Add(pdf.Id)

 End If

 Return success

End Function

Public Shared Sub SaveUsers(ByVal users As List(Of User))

 Try

 Serialization.Serialize(Of List(Of User))(users, "users.xml")

 Catch

 MessageBox.Show("Error: Saving User List Failed.")

 End Try

End Sub

Public Shared Function LoadUsers() As List(Of User)

 Dim users As List(Of User)

 Try

 users = Serialization.Deserialize(Of List(Of User))("users.xml")

 Catch

 MessageBox.Show("Error: No User database file - creating new database with sample data.")

 users = New List(Of User)()

 users.Add(New User("Bruce", "qwe", New List(Of Integer)(), 250.00))

 users.Add(New User("Joseph", "asdf", New List(Of Integer)(), 150.00))

 users.Add(New User("George", "asdf", New List(Of Integer)(), 150.00))

 Serialization.Serialize(Of List(Of User))(users, "users.xml")

 End Try

 Return users

```
End Function
End Class
End Namespace
```

PDF.vb

```
Imports System
Imports System.Collections.Generic
Imports System.Linq
Imports System.Text
Imports System.Threading.Tasks
Imports System.Windows.Forms
```

Namespace kindle_clone

Public Class Pdf

```
Public Property Id As Integer
Public Property File As String
Public Property Cost As Double
```

Public Sub New()

```
Id = 0
File = ""
Cost = 0.00
End Sub
```

Public Sub New(ByVal id As Integer, ByVal file As String, ByVal cost As Double)

```
Id = id
File = file
Cost = cost
End Sub
```

Public Shared Function LoadPdfs() As List(Of Pdf)

Dim pdfs As List(Of Pdf)

Try

pdfs = Serialization.Deserialize(Of List(Of Pdf))("pdfs.xml")

Catch

MessageBox.Show("Error: No Pdf database file - creating new database with sample data.")

pdfs = New List(Of Pdf)()

pdfs.Add(New Pdf(1, "test1.pdf", 15.00))

pdfs.Add(New Pdf(2, "test2.pdf", 35.00))

pdfs.Add(New Pdf(3, "test3.pdf", 10.00))

pdfs.Add(New Pdf(4, "test4.pdf", 75.00))

pdfs.Add(New Pdf(5, "test5.pdf", 45.00))

Serialization.Serialize(Of List(Of Pdf))(pdfs, "pdfs.xml")

End Try

Return pdfs

End Function

End Class

End Namespace

Login.vb

```
Imports System
Imports System.Collections.Generic
Imports System.ComponentModel
Imports System.Data
Imports System.Drawing
Imports System.Linq
Imports System.Text
```

Imports System.Threading.Tasks

Imports System.Windows.Forms

Namespace kindle_clone

Public Partial Class Login

Inherits Form

Private users As List(Of User)

Private pdfs As List(Of Pdf)

Public Sub New()

InitializeComponent()

users = User.LoadUsers()

pdfs = Pdf.LoadPdfs()

End Sub

Private Sub loginBtn_Click(ByVal sender As Object, ByVal e As EventArgs)

Dim username As String = userBox.Text

Dim password As String = passBox.Text

For i As Integer = 0 To users.Count - 1

If users(i).Name = username AndAlso users(i).Password = password Then

Me.Visible = False

New Form1(users, pdfs, i).ShowDialog()

Me.Close()

Exit For

End If

Next

If Me.Visible Then

MessageBox.Show("Invalid login.")

End If

End Sub

End Class

End Namespace